

## CA3T3: Design and Analysis of Algorithms

### Unit 2

- Brute Force & Exhaustive Search:
  - Introduction to Brute Force approach,
  - Selection Sort
  - Bubble Sort,
  - Sequential search,
  - Exhaustive Search,
  - Travelling Salesman Problem
  - Knapsack Problem,
- Depth First Search,
- Breadth First Search.

### Brute Force & Exhaustive Search

#### Introduction to Brute Force approach

The Brute Force Approach (also called the Exhaustive Search Method) is the simplest algorithm design technique. It solves a problem by trying every possible solution until the correct one is found.

Instead of using advanced techniques or optimizations, a brute force algorithm checks all possible cases and selects the correct or best answer.

#### Simple Definition

Brute Force is a straightforward method of solving a problem by checking every possible solution until the correct one is found.

#### Examples of Brute Force:

1. Selection sort
2. Bubble sort
3. Sequential search
4. Brute force string matching
5. Exhaustive search
  - a. Travelling Salesman problem
  - b. Knapsack problem
  - c. Assignment Problem

#### Selection Sort

- Selection sort is a simple and efficient sorting algorithm that works by repeatedly selecting the smallest element from the unsorted portion of the list and moving it to the sorted portion of the list.
- The algorithm repeatedly selects the smallest/largest element from the unsorted portion of the list.
- Swaps it with the first element of the unsorted part. This process is repeated for the remaining unsorted portion until the entire list is sorted.
- The input order does not affect the number of comparisons because the algorithm always scans the remaining unsorted portion of the array.

- **Example:** Lets consider the following array as an example:

**arr[] = {64, 25, 12, 22, 11}**

|           |           |           |           |           |
|-----------|-----------|-----------|-----------|-----------|
| <b>64</b> | <b>25</b> | <b>12</b> | <b>22</b> | <b>11</b> |
| <b>0</b>  | <b>1</b>  | <b>2</b>  | <b>3</b>  | <b>4</b>  |

- **Selection Sort Algorithm**

**Algorithm:** SelectionSort(A[0...n-1])

**Input:** An array A[0...n-1] of orderable elements.

**Output:** Array A[0...n-1] sorted in non-decreasing (ascending) order.

♦ **Algorithm: Selection Sort (A[0...n-1])**

Step 1: Repeat Step 2 to Step 5 for i = 0 to n-2

Step 2: Set min = i

Step 3: Repeat for j = i+1 to n-1

    If A[j] < A[min] then

        Set min = j

Step 4: Swap A[i] and A[min]

Step 5: End loop

Step 6: Exit

**Explanation**

- The algorithm divides the array into sorted and unsorted parts.
- In each pass/iteration, it finds the minimum element from the unsorted part.
- The minimum element is then swapped with the first element of the unsorted part.
- After the  $i^{\text{th}}$  pass, the first  $i + 1$  elements are in their correct sorted positions.

- **Mathematical Analysis of Selection Sort**

**Algorithm:** Selection Sort

**Input Size:** Number of elements in the array = n

**Basic Operation:** Comparison: A[j] < A[min]

**Number of Comparisons**

The inner loop executes:

- 1st pass: n-1 comparisons
- 2nd pass: n-2 comparisons
- 3rd pass: n-3 comparisons
- ...
- Last pass: 1 comparison

Therefore,

$$C(n) = (n - 1) + (n - 2) + (n - 3) + \dots + 2 + 1$$

Using the formula,

$$1 + 2 + 3 + \cdots + (n - 1) = \frac{n(n - 1)}{2}$$

Hence,

$$C(n) = \frac{n(n - 1)}{2}$$

### Worst Case Analysis

Number of comparisons:

$$C_{\text{worst}}(n) = \frac{n(n - 1)}{2}$$

Time Complexity:

$$O(n^2)$$

### Best Case Analysis

Even if the array is already sorted, Selection Sort still compares every remaining element to find the minimum.

Number of comparisons:

$$C_{\text{best}}(n) = \frac{n(n - 1)}{2}$$

Time Complexity:

$$O(n^2)$$

### Average Case Analysis

Number of comparisons:

$$C_{\text{average}}(n) = \frac{n(n - 1)}{2}$$

Time Complexity:

$$O(n^2)$$

### Number of Swaps

Although comparisons remain the same in all cases, the number of swaps differs.

- **Worst Case:**  $n-1$  swaps
- **Best Case:** Up to  $n-1$  swaps (or 0 unnecessary swaps if the implementation checks  $\text{min} \neq i$  before swapping)
- **Average Case:** Approximately  $n-1$  swaps

| Case         | Number of Comparisons | Time Complexity |
|--------------|-----------------------|-----------------|
| Best Case    | $\frac{n(n-1)}{2}$    | $O(n^2)$        |
| Average Case | $\frac{n(n-1)}{2}$    | $O(n^2)$        |
| Worst Case   | $\frac{n(n-1)}{2}$    | $O(n^2)$        |

**Space Complexity:**  $O(1)$

**Note:**

Selection Sort always performs exactly  $\frac{n(n-1)}{2}$  comparisons regardless of whether the input array is already sorted, reverse sorted, or randomly ordered. The only variation is in the number of swaps, not the number of comparisons.

## Bubble Sort

- **Bubble Sort** is the simplest sorting algorithm that works by repeatedly swapping the adjacent elements until the element is in sorted order.
- In each iteration last swapped element will be in sorted order
- This algorithm is not suitable for large data sets as its average and worst-case time complexity is quite high.
- Let us understand the working of bubble sort with the help of the following illustration:

**Input:** arr[] = {6, 0, 3, 5}

|   |   |   |   |
|---|---|---|---|
| 6 | 0 | 3 | 5 |
| 0 | 1 | 2 | 3 |

- **Bubble Sort Algorithm:**

**Algorithm:** BubbleSort(A[0...n-1])

**Input:** An array A[0...n-1] of orderable elements.

**Output:** Array A[0...n-1] sorted in non-decreasing order.

**Bubble Sort – Simplified Algorithm (A[0...n-1])**

i is nothing but pass

**Step 1:** Start

**Step 2:** Repeat Step 3 and Step 4 For i = 0 to n-1

**Step 3:** Repeat For j = 0 to n-i-1

**Step 4:** If  $A[j] > A[j+1]$  then  
Swap  $A[j]$  and  $A[j+1]$

**Step 5:** End inner loop

**Step 6:** End outer loop

**Step 7:** Stop

- **Bubble Sort Mathematical Analysis:**

**Step 1: Input Size**

Input size =  $n$  (number of elements in the array)

**Step 2: Basic Operation**

Comparison:  $A[j] > A[j+1]$

Swapping:  $\text{swap}(A[j], A[j+1])$

**Step 3: Number of Comparisons**

The number of comparisons is **same for all input arrays** of size  $n$ .

**Comparison Count**

$$C(n) = (n - 1) + (n - 2) + (n - 3) + \dots + 2 + 1$$

$$C(n) = \sum_{i=1}^{n-1} i$$

Using the formula,

$$\sum_{i=1}^{n-1} i = \frac{n(n-1)}{2}$$

Therefore,

$$C(n) = \frac{n(n-1)}{2}$$

Time Complexity

$$C(n) = O(n^2)$$

**Step 4: Number of Swaps**

The number of swaps depends on the order of the input elements.

**Best Case**

**Condition:** Array is already sorted.

**Example:**

10 20 30 40 50

- Comparisons are still performed.
- No elements are exchanged.

**Swaps**

$$S_{best}(n) = 0$$

**Time Complexity**

- Without optimization:  $O(n^2)$
- With optimized Bubble Sort(using a swap flag):  $O(n)$

**Worst Case**

Condition: Array is in reverse order.

Example:

50 40 30 20 10

Every comparison results in a swap.

**Swaps**

$$S_{worst}(n) = 1 + 2 + 3 + \dots + (n - 1)$$

$$S_{worst}(n) = \frac{n(n - 1)}{2}$$

Time Complexity:  $O(n^2)$

### **Average Case**

The array is randomly ordered.

Approximately half of the comparisons result in swaps.

**Average Swaps**

$$S_{avg}(n) \approx \frac{n(n - 1)}{4}$$

Time Complexity:  $O(n^2)$

### **Space Complexity**

Bubble Sort is an **in-place sorting algorithm** because it uses only one temporary variable during swapping.

$$\text{Space Complexity} = O(1)$$

| Case         | Number of Comparisons | Time Complexity    |
|--------------|-----------------------|--------------------|
| Best Case    | $\frac{n(n-1)}{2}$    | $O(n^2)$ or $O(n)$ |
| Average Case | $\frac{n(n-1)}{2}$    | $O(n^2)$           |
| Worst Case   | $\frac{n(n-1)}{2}$    | $O(n^2)$           |

## **Sequential search**

### **Definition:**

Compares successive elements of given list with given search key until either a match is encountered, or the list is exhausted without finding a match.

It is also known as Linear search.

### **How Does Linear Search Algorithm Work?**

- Every element in an array is compared with the search key element.
- If any element is found equal to the key, the search is successful and the index of that element is returned.
- If no element is found equal to the key, the search yields “No match found”.

**Sequential Search Algorithm:****Algorithm:** Sequential Search( $A[0 \dots n-1]$ , K)**Input:**An array  $A[0 \dots n-1]$  of  $n$  elements.A search key  $K$ .**Output:** Index of the key if found; otherwise -1.Let  $A[]$  be a linear array with  $n$  elements and key is an element to be searched in  $A[]$ .**Step 1:** Set  $i = 0$ **Step 2:** while ( $i < n$ ) do    if  $A[i] == \text{key}$  then        return  $i$ ;      // key found at  $i^{\text{th}}$  location     $i = i + 1$ 

end while

**Step 3:** return -1;      // key not found**Step 4:** End

- Mathematical Analysis of Sequential Search**

**Step 1: Input Size:** Input size =  $n$  (number of elements in the array)**Step 2: Basic Operation**

The basic operation is the comparison

 $A[i] == K$ **Step 3: Number of Comparisons**The number of comparisons depends on the **position of the key** in the array.**Best Case**Condition: The key element is present at the **first position**.

Example:

**Array: 25 18 40 12 60****Key: 25**

Only one comparison is needed.

**Comparisons**

$$C_{best}(n) = 1$$

**Time Complexity:  $O(1)$** **Worst Case****Condition**

- The key is at the last position, or
- The key is not present in the array.

**Example:****Array : 25 18 40 12 60****Key : 60****Or Key: 70(not present in the array )**

All elements are compared.

## Comparisons

$$C_{worst}(n) = n$$

**Time Complexity:  $O(n)$**

### Average Case

**Condition:** The key is equally likely to be at any position.

Average number of comparisons:

$$C_{avg}(n) = \frac{1 + 2 + 3 + \dots + n}{n}$$

Using

$$1 + 2 + \dots + n = \frac{n(n + 1)}{2}$$

Therefore,

$$C_{avg}(n) = \frac{n(n + 1)}{2n}$$

$$C_{avg}(n) = \frac{n + 1}{2}$$

**Time Complexity:  $O(n)$**

| Case         | Number of Comparisons | Time Complexity          |
|--------------|-----------------------|--------------------------|
| Best Case    | 1                     | <b><math>O(1)</math></b> |
| Average Case | $\frac{n+1}{2}$       | <b><math>O(n)</math></b> |
| Worst Case   | n                     | <b><math>O(n)</math></b> |

### Space Complexity

Sequential Search uses only one extra variable (the loop index).

$$\text{Space Complexity} = O(1)$$

## Exhaustive Search

**Definition:** If there is no known efficient method to solve a given problem then, test each possibility sequentially to check if there is a solution? Such an examination is called exhaustive search. Exhaustive search is simply a brute-force approach to combinatorial problems.

**Examples of exhaustive search are:**

- Travelling salesman problem
- Knapsack problem
- Assignment problem

### a) Travelling Salesman Problem

**Definition:** The problem asks to find the shortest tour through a given set of 'n' cities that visits each city exactly once before returning to the city where it started.

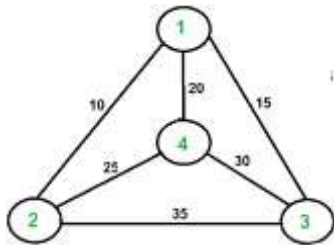
**Travelling Salesman Problem (TSP):** find shortest tour through a given set of  $n$  settings that visits each city exactly once before returning to starting city

- represent by weighted graph
  - **Vertex**  $\rightarrow$  City
  - **Edge**  $\rightarrow$  Path between two cities
  - **Edge Weight**  $\rightarrow$  Distance or cost
- with this formulation, problem becomes finding the shortest **Hamiltonian circuit** of the graph
- **Hamiltonian circuit:**

**Definition:** A Hamiltonian circuit is defined as a cycle that passes through all the vertices of the graph exactly once.

Hamilton circuit start and end with same vertex. No vertex should be repeated twice.

**Example 1: Traveling Salesman Problem(TSP)**



**Solution:**

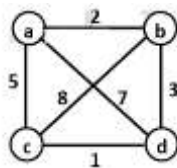
A TSP tour in the graph is 1-2-4-3-1.

The cost of the tour is  $10+25+30+15 = 80$ .

**Hence,**

minimum weight(cost) Hamiltonian Cycle:  $10 + 25 + 30 + 15 = 80$

**Example 2: Traveling Salesman Problem(TSP)**



**Solution** to small instance of the travelling salesman problem by exhaustive search.

| Tour              | Length                   |                |
|-------------------|--------------------------|----------------|
| a - b - c - d - a | $L = 2 + 8 + 1 + 7 = 18$ |                |
| a - b - d - c - a | $L = 2 + 3 + 1 + 5 = 11$ | <i>Optimal</i> |
| a - c - b - d - a | $L = 5 + 8 + 3 + 7 = 23$ |                |
| a - c - d - b - a | $L = 5 + 1 + 3 + 2 = 11$ | <i>Optimal</i> |
| a - d - b - c - a | $L = 7 + 3 + 8 + 5 = 23$ |                |
| a - d - c - b - a | $L = 7 + 1 + 8 + 2 = 18$ |                |

### Traveling Salesman Problem (TSP) Algorithm:

**Algorithm:** TravelingSalesman( $G, n, S$ )

**Input:**

- A graph  $G$  containing  $n$  cities.
- Distance (cost) between every pair of cities.
- Starting city  $S$ .

**Output:**

- The shortest tour that visits every city exactly once and returns to the starting city.
- Minimum tour cost.

Let  $G$  be a graph with  $n$  cities and  $S$  be the starting city.

**Algorithm Steps**

**Step 1:** Set  $\text{minCost} = \infty$

**Step 2:** Set  $\text{bestTour} = \text{NULL}$

**Step 3:** Generate all possible permutations of the remaining  $(n - 1)$  cities.

**Step 4:** For each generated tour, do

    Calculate the total tour cost.

    If ( $\text{tourCost} < \text{minCost}$ ) then

$\text{minCost} = \text{tourCost}$

$\text{bestTour} = \text{currentTour}$

    End If

**Step 5:** Repeat Step 4 until all possible tours are evaluated.

**Step 6:** Print  $\text{bestTour}$  and  $\text{minCost}$ .

**Step 7:** End.

### Time Complexity Analysis

For  $n$  cities, one city is fixed as the starting point. The remaining  $(n - 1)$  cities can be arranged in  $(n - 1)!$  possible ways. Each possible tour requires  $O(n)$  time to calculate its total cost.

The remaining  $(n - 1)$  cities can be arranged in:

$$(n - 1)!$$

different ways.

For each tour, we calculate the cost by visiting  $n$  cities.

Therefore:

$$T(n) = n(n - 1)!$$

Hence, the time complexity is  $O(n!)$

**Conclusion:** Time Complexity =  $O(n!)$

The exhaustive search algorithm is therefore suitable only for a **small number of cities** because the number of possible tours increases very rapidly.

### Space Complexity Analysis

The algorithm stores:

- The current tour
- The best tour
- Cost variables

The space required for storing a tour of  $n$  cities is:

## Knapsack Problem

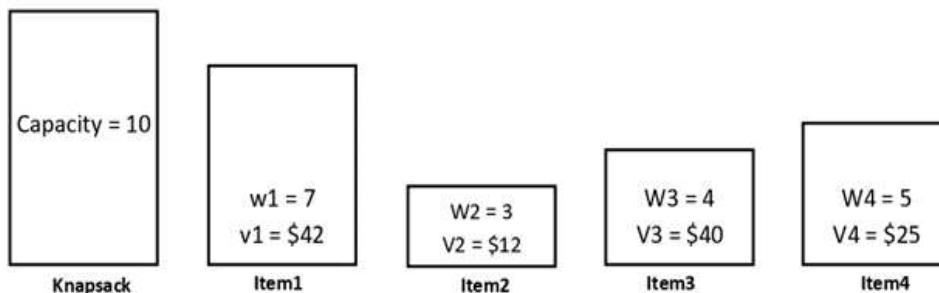
### Definition:

Given  $n$  items of known weights  $W_1, W_2, \dots, W_n$  and values/profits  $V_1, V_2, \dots, V_n$  and a knapsack of capacity  $W$ , find the most valuable subset of the items that fit into the knapsack.

- Think about a transport plane that has to deliver the most valuable set of items to a remote location without exceeding the plane's capacity.

### Example:

- Given 4 items of known weights  $W_1, W_2, W_3, W_4$  and values/profits  $V_1, V_2, V_3, V_4$  and a knapsack of capacity  $W=10$ , find the most valuable subset of the items that fit into the knapsack.



### Instance of the knapsack problem

### Solution:

Exhaustive search presents a small instance of the knapsack problem. The exhaustive-search approach to this problem leads to generating all the subsets of  $n$  items given, computing the total weight of each subset in order to identify feasible subsets and finding a subset of the largest value among them.

### Analysis:

Since the number of subsets of an  $n$ -element set is  $2^n$

Efficiency =  $O(2^n)$ .

| Subset        | Total weight | Profit (value) |
|---------------|--------------|----------------|
| $\emptyset$   | 0            | \$0            |
| {1}           | 7            | \$42           |
| {2}           | 3            | \$12           |
| {3}           | 4            | \$40           |
| {4}           | 5            | \$25           |
| {1, 2}        | 10           | \$54           |
| {1, 3}        | 11           | Not feasible   |
| {1, 4}        | 12           | Not feasible   |
| {2, 3}        | 7            | \$52           |
| {2, 4}        | 8            | \$37           |
| <b>{3, 4}</b> | <b>9</b>     | <b>\$65</b>    |
| {1, 2, 3}     | 14           | Not feasible   |
| {1, 2, 4}     | 15           | Not feasible   |
| {1, 3, 4}     | 16           | Not feasible   |
| {2, 3, 4}     | 12           | Not feasible   |
| {1, 2, 3, 4}  | 19           | Not feasible   |

Instance of the knapsack problem solutions by exhaustive search. The information about the optimal selection is in bold total weight=9, Total profit=\$65.

### **Knapsack Problem Algorithm:**

Algorithm Steps

ALGORITHM Knapsack(n, Weight, Profit, W)

#### **INPUT:**

n → Number of items

Weight[1...n] → Weights of n items

Profit[1...n] → Profits of n items

W → Capacity of the knapsack

#### **OUTPUT:**

bestSubset → Selected items

maxProfit → Maximum profit

**Step 1:** Set maxProfit = 0.

**Step 2:** Set bestSubset = NULL.

**Step 3:** Generate all possible subsets of the given n items.

**Step 4:** For each subset, do

Set totalWeight = 0.

Set totalProfit = 0.

**Step 5:** For each item in the subset, do

totalWeight = totalWeight + Weight[i]

totalProfit = totalProfit + Profit[i]

**Step 6:** Check whether the subset satisfies the knapsack capacity.

If (totalWeight ≤ W) then

    If (totalProfit > maxProfit) then

        maxProfit = totalProfit

        bestSubset = current subset

    End If

End If

**Step 7:** Repeat Steps 4 to 6 for all possible subsets.

**Step 8:** Print bestSubset.

**Step 9:** Print maxProfit.

**Step 10:** End.

### **Analysis of Knapsack Algorithm**

For n items, each item has two choices:

Include the item.

Exclude the item.

Therefore, the total number of possible subsets is:

$$2^n$$

The exhaustive search algorithm generates and examines all possible subsets.

Hence, the running time grows exponentially.

$$T(n) = O(2^n)$$

Space Complexity

If the selected subset of items is stored, space is required for at most n items.

S(n)=O(n)

**Depth First Search:**

- It proceeds in a concentric manner by visiting first all the vertices that are adjacent to a starting vertex, then all unvisited vertices two edges apart from it, and so on, until all the vertices in the same connected component as the starting vertex are visited.
- If there still remain unvisited vertices, the algorithm has to be restarted at an arbitrary vertex of another connected component of the graph.
- Data Structure used in BFS: **Queue**.

**Queue Traversal:**

- The queue is initialized with the traversals starting vertex, which is marked as visited.
- On each iteration, the algorithm identifies vertices that are adjacent to the front vertex, marks them as visited, and adds them to the queue; after that, the front vertex is removed from the queue.

**Example, Algorithm, Mathematical Analysis**  
**Check PPT**

**Breadth First Search:**

- DFS starts a graphs traversal at an arbitrary vertex by making it as visited.
- On each iteration, the algorithm proceeds to an unvisited vertex that is adjacent to the one it is currently in. If there are several such vertices, a tie can be resolved arbitrarily (like the alphabetical order).
- The process continues until a dead end- a vertex with no adjacent unvisited vertices is encountered.
- At a dead end, the algorithm back-up one edge to the vertex it came from and tries to continue visiting unvisited vertices from there.
- The algorithm eventually halts after backing up to the starting vertex, with the latter being a dead end. By then, all the vertices in the same connected component as the starting vertex have been visited.
- If unvisited vertices still remain, the depth-first search must be restarted at any one of them.
- Graph may be undirected or weighted graph.
- Data Structure used in DFS is **Stack**.

Stack uses two subscripts: Subscript1(push order) subscript2(pop order)

### **Tree edge**

- Whenever a new unvisited vertex is reached for the first time, it is attached as a child to the vertex from which it is being reached. Such an edge is called a **tree edge** because the set of all such edges forms a forest.

### **Back edge**

- Back edge is an edge  $(u, v)$  such that  $v$  is ancestor of edge  $u$  but not part of DFS tree. It is an edge leading to a previously visited vertex other than its immediate predecessor.

### **• DFS Applications**

#### **○ Check connectivity and Acyclicity of a graph**

- *In DFS forest, if the graph does not have back edges, the graph is acyclic.*
- *If there is a back edge, the graph has a cycle.*

#### **○ Finding articulation points of a graph**

- *A vertex of a connected graph is said to be its articulation point if its removal with all edges incident to it breaks the graph into disjoint pieces.*

## **Example, Algorithm, Mathematical Analysis**

### **Check PPT**

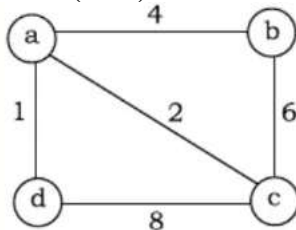
## Unit Wise Previous Year Question Papers

### 2 Marks

1. What is Brute Force Approach?
2. What is Selection Sort?
3. What is Bubble Sort?
4. What is Sequential Search?
5. What is Travelling Salesman Problem?
6. What is Hamiltonian Circuit?
7. What is Knapsack Problem?
8. What are the graph traversal methods?
9. What is DFS?
10. What is BFS?
11. Write down the formula to calculate optimal solution in 0/1 Knapsack problem.

### 5 Marks

1. Apply brute force exhaustive search approach to solve Travelling Salesman Problem(TSP).



2. Explain Selection Sort with example.
3. Write an algorithm and perform mathematical analysis for selection sort.
4. Explain Bubble Sort with example.
5. Write an algorithm and perform mathematical analysis for bubble sort.
6. Explain Sequential search with example.
7. Write an algorithm and perform mathematical analysis for sequential search.
8. Write and explain BFS algorithm with an example.
9. Elaborate the Hamiltonian Cycles problem in an Undirected Graph.
10. Illustrate the implementation of travelling salesman problem using dynamic programming.

### 10 Marks

1. Explain Brute force Approach in detail.
2. Explain Travelling Salesman Problem with algorithm and analysis in detail.
3. Explain Knapsack problem with algorithm and analysis in detail.
4. Explain BFS traversal with algorithm and analysis in detail.
5. Explain DFS traversal with algorithm and analysis in detail.